

Write a function:
 string longestWordWithPrefix(TrieNode* root, string prefix);

It should return the longest word in the trie that starts with prefix.
 If multiple words tie, return any one.

You must use a recursive helper.

Helper:
 void longestWordRec(TrieNode* node, string current, string& longest);

- string longestWordWithPrefix(TrieNode* root, string prefix)
1. Make a pointer to root
 2. Traverse trie using prefix
 3. Make a temp variable temp = ""
 4. return string

- void longestWordRec(TrieNode* node, string current, string& longest)
1. Base case if node doesn't exist
 2. Base case if long word found
 3. Loop through trie and recursively looking for longest

```
string longestWordWithPrefix(TrieNode* root, string prefix)
{
    TrieNode* arrow = root;
    string longest = ""; // temp variable
    for (int i = 0; i < prefix.length(); ++i) {
        if (!arrow->children[prefix[i]])
            return ""; // break
        arrow = arrow->children[prefix[i]]
    }
    longestWordRec(arrow, prefix, longest);
    return longest;
}
```

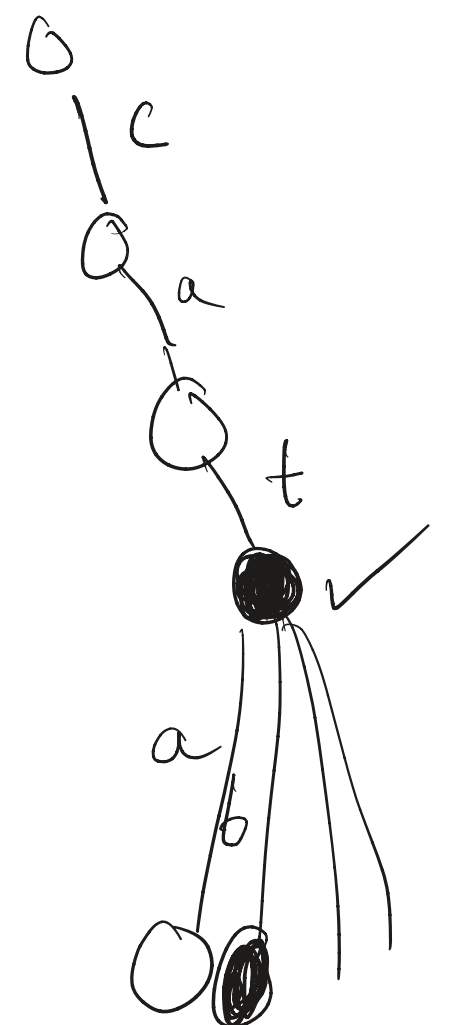
```
void longestWordRec(TrieNode* node, string current, string& best)
```

```
{
    if (!node)
        return;
    if (node->marked && current.length() > best.length())
        best = current;
    for (int i = 0; i < 256; ++i) {
        if (node->children[i]) {
            // char c = 'a' + i only if working letters
            current += i;
            longestWordRec(node->children[i], current, best);
        }
    }
}
```

Node {

next
 children

}



current += i

longestWordRec(node->children[i], current, best);

}

}